

# Technical Solution Document

## Product understanding

GuisedUp presents a feed built around genuine relationships rather than pure engagement. Users authenticate, publish a story, record lightweight interactions, discover semantically related stories, and receive a personalized ranking.

## Architecture

text  
Expo React Native app  
| Bearer-token HTTPS  
Laravel 12 API — MySQL (users, posts, interactions, Sanctum tokens)  
|  
| HTTP /embed, /search  
Flask embedding service — SentenceTransformers — ChromaDB persistent vectors

Laravel is the system of record. Chroma only stores vector ids and text vectors, so it never decides access control or supplies user-visible records directly.

## Database design

Users has many posts and interactions each post belongs to one author and has many interactions. posts.user\_id, posts.created\_at and interactions.user\_id, interactions.post\_id and interactions.type are indexed for the principal feed, relationship, and analytics queries. Foreign keys cascade to keep derived interaction rows valid.

## API design

All mutating and feed/search endpoints are under `auth:sanctum`; login returns a personal-access token and logout revokes the current token. Input is validated with Form Requests. API resources shape post responses consistently and pagination intentionally returns only `data`, `current\_page`, and `last\_page` for a simple mobile contract.

## Feed ranking logic

For a bounded recent candidate set, FeedRankingService calculates:

text  
0.40 relationship depth + 0.30 semantic similarity  
+ 0.20 authenticity score + 0.10 recency score

Relationship depth is the viewer's normalized interaction count with the author (soft-capped at 20). Semantic relevance uses the viewer's recently interacted post text to query Chroma and applies a rank-decay score. Authenticity is a bounded local text-quality heuristic. Recency uses exponential decay with a 72-hour half-life. Scores are calculated in-memory after batched relationship lookup, avoiding per-post relationship queries.

## **Vector search design**

The Flask service loads `all-MiniLM-L6-v2`, produces normalized embeddings, and stores them in a persistent local Chroma collection using cosine distance. `/embed` creates a UUID returned to Laravel; `/search` returns nearest ids. Laravel resolves those ids to authorized relational posts and preserves vector relevance ordering. If Flask is unavailable, creation remains available and search returns no vector results; this is a deliberate local-development resilience tradeoff.

## **Security**

Passwords use Laravel's hashed cast, credentials are validated server-side, Sanctum guards protected endpoints, and token revocation is scoped to the current access token. URL, post, and interaction fields have explicit validation. Secrets belong in .env, never in the mobile bundle. Production should require TLS, set CORS allowlists, rate-limit login and write routes, and run MySQL under a least-privilege application user.

## **Tradeoffs**

Candidate ranking is intentionally application-side and capped for clarity. It is adequate for an assessment but not for a very large feed. The vector service is a separate local process rather than an asynchronous queue, retaining a straightforward request path at the cost of occasional embedding latency. Interaction counts are used as a relationship proxy because no explicit follow/graph table is in scope.

## **Future improvements**

Add queued embedding jobs and retries, a durable outbox, cached/precomputed feed scores, explicit relationship edges, cursor pagination, image moderation, richer reaction metadata, observability, rate limits, and vector metadata filters. At scale, use candidate generation in SQL/vector search followed by a learned ranking model with offline relevance evaluation.

## **AI tools used**

OpenAI Codex assisted with project scaffolding, implementation, documentation, and test-oriented review. Design choices and final integration remain explicit in the repository.